

VIBEVOICE-ASR-STREAMING Technical Report

Yujie Tu^{2,*}, Zhiliang Peng¹, Jianwei Yu¹, Yaoyao Chang¹
Songchen Xu³, Li Dong¹, Wenhui Wang¹, Zilong Wang¹
Zehua Wang¹, Yan Xia¹, Jiajun Zhang², Xie Chen³, Furu Wei^{1,◇}

¹Microsoft Research ²University of Chinese Academy of Sciences ³Shanghai Jiao Tong University

<https://aka.ms/GeneralAI>

Traditional speaker-attributed ASR systems treated ASR and speaker diarization as two separate tasks. Recently, end-to-end models such as VibeVoice-ASR have unified the two tasks within a single model. However, existing unified models still mainly support offline recognition, making it difficult to meet the low-latency requirements of real-time voice assistants and agents. To tackle this issue, we present VIBEVOICE-ASR-STREAMING, one of the first LLM-based end-to-end approaches to streaming speaker-attributed ASR. It interleaves fixed-size audio chunks, a small amount of lookahead audio and previous text. This allows the model to produce “who said what” as speech arrives, without a separate diarization stage. For transcription accuracy, our 7B model achieves the lowest average WER/CER across five evaluation sets. For speaker attribution, it achieves the best or tied-best on 12 of 13 evaluation settings. We release the 1.5B and 7B model weights together with inference code.

 **Code:** github.com/microsoft/VibeVoice
 **Demo:** [microsoft/VibeVoice-ASR-Streaming](https://microsoft.com/VibeVoice-ASR-Streaming)
 **Checkpoint:** [microsoft/VibeVoice-Collection](https://microsoft.com/VibeVoice-Collection)

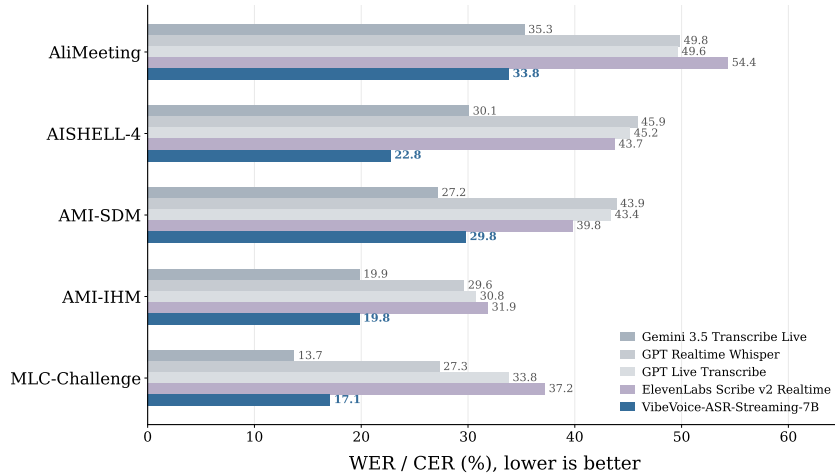


Figure 1: Recognition error of VIBEVOICE-ASR-STREAMING-7B and four deployed streaming ASR systems on the four meeting benchmarks and MLC-Challenge. AliMeeting and AISHELL-4 are evaluated with CER, while AMI-SDM and AMI-IHM are evaluated with WER. For MLC-Challenge, Japanese and Korean are evaluated with CER and the remaining seven languages with WER; the reported value is the macro average over the nine evaluated languages.

*Work done during Yujie Tu’s internship at Microsoft Research. ◇ Contact person: fuwei@microsoft.com.

1 Introduction

Streaming speaker-attributed ASR must output both the words and their speaker labels as the conversation unfolds. Each sentence is attributed to a speaker when it is emitted, rather than after the recording ends. This capability has become increasingly valuable as speech interaction has attracted growing attention in recent years. When a voice agent is in a conversation with more than one person, it has to identify who is speaking while they are still speaking in order to process the information correctly and reduce its response latency.

Three lines of work bear on this. LLM-based recognizers now transcribe long recordings and assign speakers in a single generative pass, including VibeVoice-ASR [PYC⁺26], MOSS Transcribe Diarize [YLY⁺26], SoulX-Transcriber [DLL⁺26], and SpeakerLM [YCD⁺25], but they read the whole recording before emitting output. A second line makes LLM-based ASR streamable: BE-STOW [CHH⁺24] casts inference as a read-write problem, while SpeechLLM-XL [JKZ⁺24] and Uni-ASR [XTH⁺26] consume audio in chunks and carry preceding speech-text context forward, establishing the basic recipe of incremental input with retained context — for single-speaker transcription. A third line makes multi-talker recognition low-latency, from SURT [LKL⁺21, RPK⁺23] and t-SOT [KWW⁺22a], which serialize overlapping talkers, to systems that attach speaker identity at low latency through token-level speaker embeddings [KWW⁺22b], an auxiliary speaker branch [RWM⁺24], or online diarization cascaded with a recognizer [HLS⁺21, LSL⁺24, PMD⁺25, MPW⁺25]. Concurrent Speech-LLM systems target the same setting [SXF⁺25, PCL⁺26].

What the two streaming lines each leave open is the requirement speaker attribution places on retained history. For ordinary ASR, preceding context mainly helps linguistic and acoustic modeling; speaker attribution asks more of it. A speaker who appears in the current chunk may have first appeared several minutes earlier and must still receive the same label, so the retained history does not merely help: it is what fixes the speaker identities of the conversation. The streaming Speech-LLM recipe naturally carries this history forward, but has so far been developed for single-speaker transcription. Streaming multi-talker systems, by contrast, usually require additional speaker-related components instead of producing speaker-attributed transcripts directly from a single model.

This report presents VIBEVOICE-ASR-STREAMING, which meets both requirements with one model rather than two components. Following previous streaming Speech-LLMs [JKZ⁺24, XTH⁺26], incoming audio and generated speaker-attributed text are interleaved, so that future acoustic context is bounded by the chunk contract while the accumulated speech, transcription, and speaker history stays in context, and diarization never becomes a stage of its own. Each chunk is followed by a fixed 4-frame (0.5-second) lookahead. We release 1.5B and 7B model weights for the 22-frame (2.9-second) chunk configuration, with an expected speaker-attribution latency of 2.00 seconds. Across four meeting conditions and nine languages of MLC-Challenge, the 7B 22-frame configuration achieves the best or tied-best speaker-attribution error on 12 of 13 settings, while also attaining the best overall recognition-only mean among the compared streaming systems.

This report contributes:

- one of the first investigations of end-to-end LLM-based streaming speaker-attributed ASR, showing that interleaved speech-text generation can support long-form streaming recognition with strong recognition and speaker-attribution performance; we release 1.5B and 7B model weights together with inference code;
- a thorough study of the key design choices for LLM-based speaker-attributed streaming ASR, including chunk size, lookahead, model scale, and speaker-label placement, together with detailed comparisons against the non-streaming model and deployed streaming systems, as well as serving-cost analysis over long recordings.

Section 2 reviews related work. Section 3 describes the architecture and streaming formulation, the training data, and the training route. Section 4 reports the main comparison against streaming systems, Section 5 the ablations, and Section 6 the limitations.

2 Related Work

Long-form speaker-attributed ASR with LLMs. Recent large language model (LLM)-based speech recognition systems have significantly improved long-form and multi-speaker transcription.

VibeVoice-ASR [PYC⁺26] supports single-pass processing of up to 60 minutes of audio and jointly models transcription and speaker information within a unified generative framework. MOSS Transcribe Diarize [YLY⁺26] further extends end-to-end speaker-attributed transcription with a 128k context window and supports recordings of up to 90 minutes. SoulX-Transcriber [DLL⁺26] improves speaker discrimination and transcription robustness through speaker-aware continuous pre-training and supervised fine-tuning, and SpeakerLM [YCD⁺25] unifies diarization and recognition in a multi-modal LLM with a flexible speaker registration mechanism. All of these read the whole recording before emitting output.

Streaming LLM-based ASR. Several studies have explored how LLM-based ASR can operate in a streaming manner. BESTOW [CHH⁺24] formulates streamable Speech-LLM inference as a read-write problem. SpeechLLM-XL [JKZ⁺24] processes speech in configurable chunks and autoregressively generates the corresponding text while carrying preceding speech-text context forward. Uni-ASR [XTH⁺26] further develops a unified streaming and non-streaming LLM-based ASR framework with context-aware training across chunks. These works establish the basic recipe for LLM-based streaming ASR: acoustic input is consumed incrementally, while previously accumulated context is retained for subsequent recognition.

Streaming multi-talker and speaker-attributed recognition. Streaming multi-talker recognition predates the Speech-LLM era. SURT [LKL21, RPK23] places an unmixing module in front of a transducer, and t-SOT [KWW⁺22a] serializes multi-talker tokens onto a single branch by emission time; in both, the output index tracks overlap and emission order rather than a speaker. Speaker-attributed variants add the missing identity constraint through an extra component: token-level speaker embeddings decoded alongside t-SOT [KWW⁺22b], or a speaker branch inside the transducer [RWM⁺24]. A parallel line keeps diarization a separate module but makes it online, from streaming EEND [HLS21, LSL24] to Sortformer [PMD⁺25] and Streaming Sortformer [MPW⁺25], whose arrival-ordered speaker cache is cascaded with a streaming recognizer. On the Speech-LLM side, JEDIS-LLM [SXF⁺25] and G-STAR [PCL⁺26] attach a speaker cache to a long-audio recognizer, though G-STAR reports chunk-wise decoding rather than a streaming deployment.

3 Method

3.1 Architecture and Streaming Formulation

Figure 2 presents the architectural overview of VIBEVOICE-ASR-STREAMING. Built on VibeVoice-ASR [PYC⁺26], VIBEVOICE-ASR-STREAMING extends long-form speaker-attributed transcription to streaming inference. Speech is encoded by the pre-trained dual tokenizers of VibeVoice [PYW⁺25], of which only the encoder halves are used. The Acoustic tokenizer follows the σ -VAE design of [SBW⁺24] and applies a hierarchical, cumulative $3,200\times$ downsampling to the 24-kHz waveform; the Semantic tokenizer operates at the same rate and yields deterministic features aligned with textual content. The two therefore provide spectral detail and linguistic content on a common temporal grid. Their representations are concatenated along the feature dimension and projected into the embedding space of a Qwen2.5 [YYZ⁺24] LLM backbone for speaker-attributed ASR. At 24 kHz, this corresponds to one latent frame every 133.3 ms, or 7.5 frames per second. Chunk size and lookahead are therefore specified in latent frames, making every setting in this report a multiple of 133.3 ms.

Following previous streaming Speech-LLMs [JKZ⁺24, XTH⁺26], we organize incoming speech and generated text as an interleaved sequence:

$$[X_1, Y_1, X_2, Y_2, \dots], \quad (1)$$

where X_k denotes the k -th speech chunk and Y_k denotes the corresponding speaker-attributed transcription. Unlike independent chunk-wise decoding, previously observed speech and generated text remain in the LLM context when subsequent audio arrives, so each chunk is decoded against the conversation history accumulated before it.

Retaining this history is a condition of the task rather than an optimization. A system that discards the history has to reintroduce it elsewhere, as an external embedding store, a speaker cache, or an offline clustering pass, which reinstates the separate stage this formulation removes.

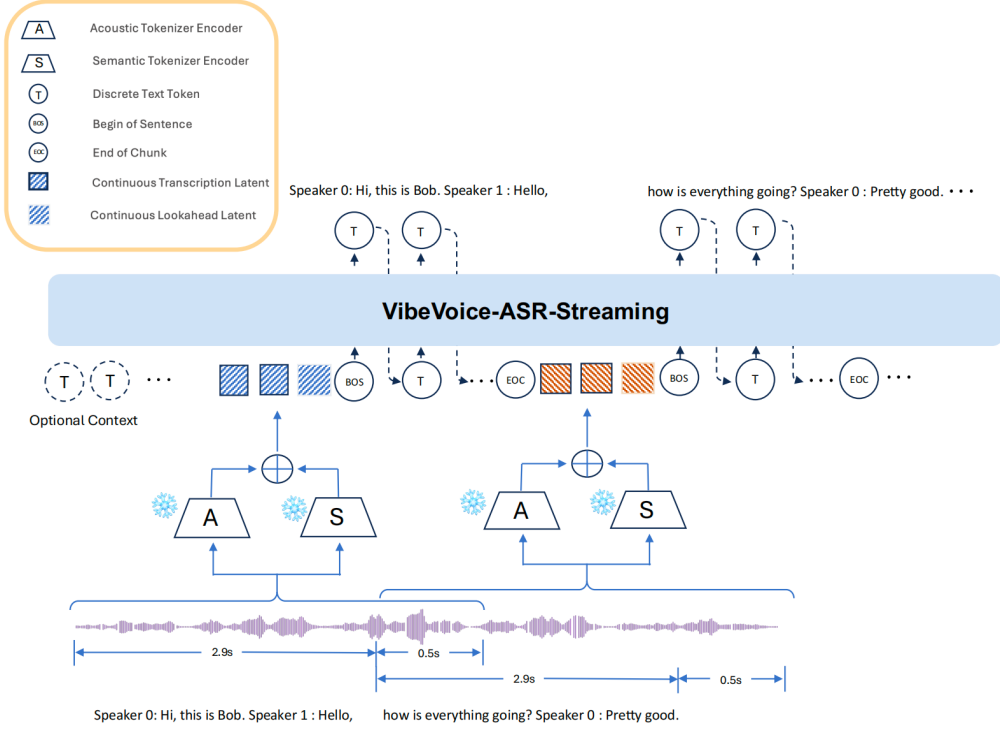


Figure 2: Architecture of VIBEVOICE-ASR-STREAMING. Speech chunks X_k and speaker-attributed text chunks Y_k are interleaved in a single autoregressive context, and each chunk is followed by a fixed $L = 4$ -frame (0.5 s) lookahead before its text is generated.

To provide limited future acoustic evidence near chunk boundaries, we introduce a fixed lookahead. Before generating the transcription associated with each chunk, the model reads an additional $L = 4$ latent frames:

$$T_{\text{lookahead}} = 4 \times \frac{3200}{24000} \approx 0.5 \text{ s.} \quad (2)$$

We evaluate two chunk configurations under this lookahead: 15 latent frames, corresponding to exactly 2.0 s of audio per chunk, and 22 latent frames, corresponding to 2.9 s per chunk.

VIBEVOICE-ASR-STREAMING formulates ASR and speaker attribution as a single autoregressive generation task and directly produces *who said what*. After receiving the current speech chunk together with its lookahead, text generation starts as soon as the audio span is closed by the speech-end token `<|object_ref_end|>`.

Let $\tilde{X}_k$ denote the current speech chunk X_k together with its L -frame lookahead. Formally, for the text sequence $Y_k = (y_{k,1}, \dots, y_{k,N_k})$ associated with chunk X_k , we have

$$p(Y_k \mid X_{< k}, \tilde{X}_k, Y_{< k}) = \prod_{j=1}^{N_k} p(y_{k,j} \mid X_{< k}, \tilde{X}_k, Y_{< k}, y_{k,< j}). \quad (3)$$

Here, $X_{< k}$ denotes the previously observed speech chunks, while $\tilde{X}_k$ contains the current chunk and the L future latent frames used as lookahead. Each Y_k ends with a special `<|text_chunk_end|>` token. Since the input does not specify how long a chunk’s transcription should be, the model must decide when to emit this token.

The token is supervised at every chunk boundary, including those with empty target text, and its emission hands control back to the audio stream. After Y_k is generated, the next speech chunk X_{k+1} is appended to the same autoregressive sequence and decoding continues.

VIBEVOICE-ASR-STREAMING also retains the contextual prompting capability of VibeVoice-ASR [PYC⁺26]. Optional context, including names, technical terms, abbreviations, and other hotwords, can be provided before decoding and remains accessible throughout the streaming session.

Output format. Each Y_k is a sequence of speaker-labeled utterances, so concatenating the per-chunk outputs already yields the speaker-attributed transcript. Speakers are identified by ordinal labels assigned in order of first appearance, and a label introduced in an early chunk is reused whenever that speaker is recognized again. Because the transcript is serialized, simultaneous speech is emitted as consecutive labeled segments rather than as parallel streams; Section 6 discusses the consequences. Appendix A gives the exact label syntax and a verbatim decoding trace.

Keeping the history uncompressed has a cost that grows linearly with recording length. Inference uses the same chunk and lookahead contract as training, and the released checkpoints target recordings of up to eight minutes.

3.2 Training Data

All training recordings, real and synthetic alike, are prepared the same way: word-level timing is obtained by running Qwen3-ForcedAligner-0.6B [Qwe26] over the recording, and the reference transcript is then split into per-chunk targets by the rule Appendix A states.

Part of the mixture is synthesized rather than collected, to improve robustness to multi-speaker acoustic conditions and specialized vocabulary. We generate meeting-style multi-speaker conversations with domain-specific terminology and proper nouns inserted into the dialogue, keeping spoken and written forms separate: the spoken form drives speech synthesis while the written form is retained as the ASR target, so numbers, abbreviations, and technical terms are spoken naturally but transcribed canonically. The synthesized speech then receives waveform-level augmentation: speakers are overlapped, and the mixture is convolved with room impulse responses, which apply room reverberation and microphone response in a single step. Speaker labels and alignment are updated alongside the waveform so that the supervision survives augmentation. This yields 50,884 recordings totaling 4,519.6 hours of augmented multi-speaker training speech.

3.3 Training Route

We train VIBEVOICE-ASR-STREAMING in three stages that differ in how training samples are constructed rather than in the model or the training objective.

Stage 1: non-streaming training. The model is first trained in the offline speaker-attributed setting, where the complete recording is visible before the transcription is generated. This stage establishes the basic multi-speaker recognition and speaker-attribution ability without any streaming constraint.

Stage 2: streaming pre-training. Starting from the Stage-1 checkpoint, we switch the sample construction to the interleaved form of Section 3.1: each recording is segmented into chunks, every chunk is paired with its own speaker-attributed transcription, and the fixed lookahead is appended before the corresponding text is generated. Nothing else changes: the architecture, the set of trainable modules, and the autoregressive objective are identical to Stage 1. The model therefore only has to adapt to bounded future context instead of relearning speaker-attributed transcription from scratch.

Stage 3: streaming fine-tuning. The streaming model is finally fine-tuned under the same interleaved formulation to obtain the reported systems. Stage 2 draws on a subset of the Stage-1 corpus, roughly 420,000 hours of English and Chinese speech, and its job is to make the streaming format the model’s normal operating condition. Stage 3 switches to a much smaller curated mixture, about 13,000 hours drawn from public training splits and from the synthetic multi-speaker data of Section 3.2, and its job is to settle the behavior a user actually experiences: transcription conventions, consistent speaker labeling, and reliable hotword following. Optimizer settings and run scale are given in Appendix A.

Each streaming configuration is initialized from the non-streaming checkpoint of the same scale, avoiding full training from scratch. The chunk size is fixed throughout Stages 2 and 3, and the 15- and 22-frame configurations are trained independently.

Table 1: Recognition-only streaming ASR results on the four meeting benchmarks and MLC-Challenge. For all online APIs, audio is streamed using 2.9-s chunks, matching the chunk size of VIBEVOICE-ASR-STREAMING. Best results are shown in **bold** and second-best underlined. Chinese, Japanese, and Korean are evaluated with CER, and the remaining languages with WER. The final row reports the mean over the four meeting benchmarks and the MLC-Challenge macro average.

Benchmark	Language	Gemini 3.5 Transcribe Live ¹	GPT Realtime Whisper ²	GPT Live Transcribe ³	ElevenLabs Scribe v2 Realtime ⁴	VIBEVOICE-ASR-STREAMING 7B
		WER/CER	WER/CER	WER/CER	WER/CER	WER/CER
MLC-Challenge	English	8.40	20.66	21.88	22.75	<u>8.44</u>
	French	<u>19.48</u>	27.90	37.83	32.98	16.42
	German	16.96	33.25	37.35	48.68	<u>21.83</u>
	Italian	12.98	28.41	35.72	35.98	<u>17.00</u>
	Japanese	16.54	36.50	33.48	33.17	<u>27.85</u>
	Korean	<u>9.59</u>	24.98	32.18	31.53	9.09
	Portuguese	20.83	38.86	53.46	59.20	<u>28.15</u>
	Russian	9.66	20.55	29.38	46.78	<u>15.67</u>
	Spanish	9.13	14.88	23.13	23.79	<u>9.31</u>
	Average	13.73	27.33	33.82	37.21	<u>17.09</u>
AliMeeting	Chinese	<u>35.29</u>	49.84	49.64	54.35	33.83
AISHELL-4	Chinese	<u>30.09</u>	45.86	45.17	43.74	22.76
AMI-SDM	English	27.18	43.90	43.38	39.81	<u>29.81</u>
AMI-IHM	English	<u>19.85</u>	29.64	30.75	31.86	19.83
Average	Mix	<u>25.23</u>	39.31	40.55	41.39	24.66

4 Results

All models use the frozen Acoustic and Semantic tokenizer encoders and the trainable Qwen2.5 LLM backbone of Section 3.1, and differ only in backbone scale, 1.5B and 7B. Each scale is trained at two chunk sizes, 22 latent frames (2.9 s of audio) and 15 latent frames (2.0 s), under the same fixed 4-frame (0.5 s) lookahead; unless otherwise stated the reported results use the 7B model with 22-frame chunks.

Datasets. We evaluate on the Chinese meeting corpora AISHELL-4 [FCL⁺21] and AliMeeting [YZF⁺22], on AMI [CAB⁺05] in both its individual-headset (AMI-IHM) and single-distant-microphone (AMI-SDM) conditions, and on nine languages of the conversational benchmark MLC-Challenge [MGS⁺25]: English, French, German, Italian, Japanese, Korean, Portuguese, Russian, and Spanish. The benchmark itself covers more languages than these, but the forced aligner of Section 3.2 does not, so the remaining languages are absent from training and we do not report them. All evaluation recordings are capped at 480 seconds to match the maximum session length supported by the released checkpoints. Single-speaker results are additionally reported on AISHELL-1 [BDN⁺17], LibriSpeech [PCPK15] test-clean and test-other, and GigaSpeech [CCW⁺21]. Several of these corpora also contribute to training, but only through their official training splits; no evaluation utterance appears in any training mixture.

Metrics. We follow the MeetEval [vNBDHU23]⁵ protocol and report word error rate (WER), which ignores speaker attribution and so reflects recognition quality alone, and concatenated minimum-permutation WER (cpWER), which concatenates the hypotheses and references belonging to each speaker and takes the minimum error over speaker permutations. Chinese, Japanese, and Korean are scored at the character level for every system alike, as CER and cpCER, and columns headed WER and cpWER carry those values on any row or language so scored, including inside the MLC-Challenge average. Speaker-attribution latency is the delay between a word being spoken and

¹<https://ai.google.dev/gemini-api/docs/models/gemini-3.5-transcribe>

²<https://developers.openai.com/api/docs/models/gpt-realtime-whisper>

³<https://developers.openai.com/api/docs/models/gpt-live-transcribe>

⁴<https://elevenlabs.io/docs/api-reference/speech-to-text/v-1-speech-to-text-realtime>

⁵<https://github.com/fgnt/meeteval>

Table 2: Overall streaming speaker-attributed ASR results for the 7B model with 22-frame (2.9 s) chunks. Chinese, Japanese, and Korean are evaluated with CER and cpCER, and the remaining languages with WER and cpWER. For Google STT, D_0 uses the speaker label at first emission, while D_{fin} uses the final revised label after the full recording has been processed. Best results are shown in **bold** and second-best underlined, ranked over the systems that report the metric. A dash marks a language or corpus the system does not support.

		Azure CT ⁶		Google STT ⁷			VIBEVOICE-ASR-STREAMING 7B	
Dataset	Language	WER/CER	cpWER/cpCER	WER/CER	cpWER/cpCER	cpWER/cpCER	WER/CER	cpWER/cpCER
Avg. latency		8.21 s		–	9.12 s (D_0)	51.06 s (D_{fin})	2.00 s	
MLC-Challenge	English	9.31	23.02	10.57	60.57	28.40	8.44	11.99
	French	20.55	33.58	–	–	–	16.42	21.50
	German	21.12	27.17	–	–	–	21.83	24.68
	Italian	13.67	21.53	–	–	–	17.00	21.20
	Japanese	15.04	33.01	–	–	–	27.85	33.01
	Korean	10.32	23.97	–	–	–	9.09	23.22
	Portuguese	22.76	36.11	–	–	–	28.15	38.25
	Russian	14.99	28.11	–	–	–	15.67	19.47
	Spanish	11.12	17.00	–	–	–	9.31	11.46
AVERAGE		15.43	27.06	–	–	–	17.09	22.75
AliMeeting	Chinese	29.40	52.25	–	–	–	33.83	39.80
AISHELL-4	Chinese	24.03	32.51	–	–	–	22.76	28.70
AMI-SDM	English	33.27	41.40	32.11	73.63	46.65	29.81	39.01
AMI-IHM	English	24.01	31.90	23.07	71.48	40.15	19.83	27.48

67

its speaker-attributed transcription settling: an *expected algorithmic* delay $C/2 + T_{\text{lookahead}}$ for VIBEVOICE-ASR-STREAMING, with C the chunk duration, giving 2.00 s at 22 frames and 1.53 s at 15, and a *measured wall-clock* mean for the cloud services. Appendix B gives the scoring and measurement details.

Compared systems. For recognition-only comparison, Figure 1 and Table 1 additionally include Gemini 3.5 Transcribe Live¹, GPT Realtime Whisper², GPT Live Transcribe³, and ElevenLabs Scribe v2 Realtime⁴. For speaker-attributed recognition, we compare against Microsoft Azure ConversationTranscriber (Azure CT) and Google Cloud Speech-to-Text (Google STT). Google STT revises speaker labels retroactively, so we report two operating points: D_0 , using each speaker label when it is first emitted, and D_{fin} , using the final label after the entire recording has been processed.

Results. Figure 1 summarizes recognition error on the four meeting benchmarks and the macro-averaged MLC-Challenge result, while Table 1 provides the full per-language breakdown. VIBEVOICE-ASR-STREAMING is best on AISHELL-4, AliMeeting, and AMI-IHM, while Gemini 3.5 Transcribe Live is best on AMI-SDM. The five-set mean is 24.66 for VIBEVOICE-ASR-STREAMING, compared with 25.23 for Gemini 3.5 Transcribe Live, 39.31 for GPT Realtime Whisper, 40.55 for GPT Live Transcribe, and 41.39 for ElevenLabs Scribe v2 Realtime.

Across the 13 speaker-attributed settings in Table 2, VIBEVOICE-ASR-STREAMING gives the best or tied-best cpWER/cpCER on 12, improving over Azure CT by 2.39 to 12.45 points on the four meeting benchmarks and taking the best or tied-best value on eight of the nine MLC-Challenge languages. It commits far earlier than the cloud services, after an expected 2.00 s against a measured 8.21 s for Azure CT and 9.12 s for Google STT, whose labels are still being revised tens of seconds later.

Single-speaker short-form audio is not what this model is built for, and it wins no individual test set in Table 3. It nonetheless stays close to the strongest system on every set: with 22-frame chunks it is second on AISHELL-1 and on both LibriSpeech splits, and its four-set mean is level with the best. The 22-frame configuration beats the 15-frame one on all four sets.

⁶<https://learn.microsoft.com/en-us/azure/ai-services/speech-service/get-started-stt-diarization>

⁷<https://docs.cloud.google.com/speech-to-text/docs/multiple-voices>

Table 3: Single-speaker recognition, reported as a class check rather than as a competitive claim. CER for AISHELL-1, WER for the others; the final column is the uniform mean over the four sets. **Bold** marks the lowest value in each column and underline the second lowest.

System	AISHELL-1	LS test-clean	LS test-other	GigaSpeech	Avg.
VIBEVOICE-ASR-STREAMING-7B (15 frm)	4.81	2.68	7.40	10.60	6.37
VIBEVOICE-ASR-STREAMING-7B (22 frm)	<u>4.01</u>	<u>2.33</u>	6.49	10.20	5.76
Nemotron-3.5-ASR 0.6B [NV126]	13.56	3.02	6.80	11.76	8.79
Voxtral-Mini-4B-Realtime [Mis26]	9.56	1.88	4.10	<u>10.11</u>	6.41
X-ASR [Gil26]	3.61	2.96	6.85	9.64	<u>5.77</u>

All rows are our own measurements under a single normalization. X-ASR is evaluated in its chunk-1920ms streaming configuration, Voxtral-Mini-4B-Realtime at `transcription_delay_ms=2400` (2.4 s), its longest configurable delay, and Nemotron-3.5-ASR at `att_context_size=[56, 13]` (1.12 s), in each case the released setting closest to our chunk sizes; Nemotron is additionally given the zh-CN language identifier on AISHELL-1 and en-US elsewhere, side information our own rows do not receive.

5 Experiments

Cost of the streaming conversion. Table 4 scores VIBEVOICE-ASR-STREAMING-7B with 22-frame chunks against the non-streaming VibeVoice-ASR checkpoint it is initialized from, scored identically. WER/CER rises by 0.75 to 3.53 points, cpWER/cpCER by 5.13 to 6.67 on every benchmark; Because cpWER/cpCER reflects both recognition and speaker assignment, its larger degradation than WER/CER suggests an additional loss associated with speaker attribution.

Table 4: Cost of the streaming conversion: VIBEVOICE-ASR-STREAMING-7B with 22-frame chunks against the non-streaming VibeVoice-ASR checkpoint it is initialized from, scored identically. Δ is streaming minus non-streaming, so positive values are degradations.

Dataset	Lang.	Non-streaming		VIBEVOICE-ASR-STREAMING		Δ	
		WER	cpWER	WER	cpWER	WER	cpWER
AISHELL-4	Chinese	19.87	22.03	22.76	28.70	+2.89	+6.67
AliMeeting	Chinese	32.15	34.12	33.83	39.80	+1.68	+5.68
AMI-IHM	English	19.08	20.98	19.83	27.48	+0.75	+6.50
AMI-SDM	English	26.28	33.88	29.81	39.01	+3.53	+5.13
MLC-Challenge	Mix	14.08	17.62	17.09	22.75	+3.01	+5.13

Chunk size and model scale. Under an identical 4-frame lookahead, enlarging the chunk from 15 to 22 latent frames improves both metrics on every benchmark and at both scales (Table 5): on the five-set mean it is worth 1.46 WER/CER and 4.06 cpWER/cpCER at 7B, and 1.31 and 2.91 at 1.5B. Scale acts the same way: at a fixed chunk size, moving from 1.5B to 7B lowers the mean cpWER/cpCER by 12.76 points at 22 frames and 11.61 at 15, against 4.69 and 4.54 of WER/CER. Both factors therefore have a stronger effect on speaker attribution than on transcription — suggesting that longer chunks and larger backbones mainly provide richer speaker evidence rather than lexical evidence. Since 15 frames reduces the expected latency from 2.00 s to 1.53 s, the two chunk settings offer different trade-offs between latency and accuracy.

Speaker-label placement. VIBEVOICE-ASR-STREAMING emits the speaker label before the text of the segment it belongs to, which makes the label available from the first token of a segment. Table 6 compares the two placements under an otherwise identical configuration: 7B backbone, 22-frame chunks, 4-frame lookahead, same data. Neither metric favors the tail: the two placements land on the same mean cpWER/cpCER, 31.55 against 31.56, and the tail is no better on WER/CER either, trailing the head by 0.98 on the five-set mean. Within the margin these measurements carry, the two placements are best read as equivalent in accuracy.

Table 5: Effect of chunk size and model scale under an identical 4-frame lookahead, giving expected latencies of 2.00 s at 22 frames and 1.53 s at 15. **Bold** marks the better chunk size for each model.

Dataset	Language	VIBEVOICE-ASR-STREAMING-7B				VIBEVOICE-ASR-STREAMING-1.5B			
		22 frm		15 frm		22 frm		15 frm	
		WER	cpWER	WER	cpWER	WER	cpWER	WER	cpWER
AISHELL-4	Chinese	22.76	28.70	24.62	32.64	28.48	39.56	30.33	42.42
AliMeeting	Chinese	33.83	39.80	34.75	41.66	39.18	48.87	39.34	51.69
AMI-IHM	English	19.83	27.48	20.82	35.40	22.85	45.15	23.39	46.48
AMI-SDM	English	29.81	39.01	31.88	42.49	34.43	56.83	35.90	59.49
MLC-Challenge	Mix	17.09	22.75	18.52	25.85	21.81	31.15	24.33	36.01
Average		24.66	31.55	26.12	35.61	29.35	44.31	30.66	47.22

That the tail speaker label buys nothing runs against what a cascaded system would predict. Where attribution is a purely acoustic decision — embed the speech, cluster the embeddings — a label is only as reliable as the amount of speech it was estimated from, and deferring it is precisely how such a system improves. Google STT measures the size of that effect: revising its labels once the whole recording is available lowers cpWER by 27.0 to 32.2 points against the label it first emits (Table 2). An end-to-end model does not decide the same way. It is producing the transcript at the same time, so who is speaking is settled from lexical and conversational evidence as much as from voice, and, as a human listener does, it can settle it within the two to three seconds a chunk contains rather than needing the segment to finish.

Table 6: Speaker-label placement: emitting the label before the segment text (head, the released configuration) against emitting it after (tail). Both runs are 7B with 22-frame chunks, a 4-frame lookahead, and the same data; Δ is tail minus head.

Dataset	Language	Head (released)		Tail		Δ	
		WER	cpWER	WER	cpWER	WER	cpWER
AISHELL-4	Chinese	22.76	28.70	24.93	28.96	+2.17	+0.26
AliMeeting	Chinese	33.83	39.80	37.72	40.78	+3.89	+0.98
AMI-IHM	English	19.83	27.48	20.13	28.22	+0.30	+0.74
AMI-SDM	English	29.81	39.01	30.65	36.52	+0.84	-2.49
MLC-Challenge	Mix	17.09	22.75	14.79	23.32	-2.30	+0.57
Average		24.66	31.55	25.64	31.56	+0.98	+0.01

Lookahead depth. Lookahead (sometimes called right context) supplies no context that a later chunk would not eventually provide; it only lets the model read across a chunk boundary before committing to the text of that chunk. Table 7 varies L over 0, 2, and 4 frames at a fixed 22-frame chunk with everything else held constant, the $L = 4$ column being the released system; both metrics improve strictly with L on all five benchmarks. The two steps cost the same 0.267 s of expected latency: averaged over the benchmarks the first pair of frames is worth 1.27 WER/CER and 1.80 cpWER/cpCER, the second pair 1.24 and 2.54, so speaker attribution is still accelerating at $L = 4$ and the released configuration reads four frames rather than fewer.

Real-time factor. The expected algorithmic latency assumes a chunk is decoded before the next one is complete, so we check it where there is least room, the 7B model at 15 frames. Decoding a chunk costs 146 to 208 ms against a 2000 ms chunk, so the real-time factor stays at or below 0.104 (Table 8).

6 Conclusion and Limitations

VIBEVOICE-ASR-STREAMING is a streaming speaker-attributed ASR framework built on VibeVoice-ASR [PYC⁺26]. Interleaving incoming speech chunks with autoregressively generated speaker-attributed text produces *who said what* in a single pass, without a separate diarization stage.

Table 7: Lookahead ablation on the 7B model at a fixed 22-frame chunk. L is the number of future latent frames read before a chunk’s text is generated; expected latency is $C/2 + L \times 133.3$ ms, so each step of the sweep costs the same 0.267 s. The $L = 0$ columns have no predecessor, hence the dashes in the Δ row.

Dataset	Language	$L = 0$ (1.47 s)		$L = 2$ (1.73 s)		$L = 4$ (2.00 s)	
		WER	cpWER	WER	cpWER	WER	cpWER
AISHELL-4	Chinese	26.07	31.98	24.11	29.16	22.76	28.70
AliMeeting	Chinese	37.02	45.02	35.06	41.07	33.83	39.80
AMI-IHM	English	21.83	34.75	20.97	34.34	19.83	27.48
AMI-SDM	English	33.39	44.05	32.09	43.01	29.81	39.01
MLC-Challenge	Mix	17.60	23.61	17.31	22.85	17.09	22.75
Average		27.18	35.88	25.91	34.09	24.66	31.55
Δ vs. previous column		–	–	–1.27	–1.80	–1.24	–2.54

Table 8: Measured serving cost of the 7B 15-frame configuration on one A100 80GB PCIe under vLLM in bfloat16, batch size one; median of three runs on AMI-IHM audio, the two longest inputs obtained by tiling a 237 s recording. Per-chunk compute stays well below the 2000 ms chunk at every length.

Audio (s)	Chunks	Decode (s)	RTF	ms / chunk
30	15	2.90	0.097	193
60	30	4.83	0.081	161
120	60	8.78	0.073	146
237	119	20.13	0.085	169
360	180	32.54	0.090	181
480	240	49.94	0.104	208

On recognition-only ASR, our 7B model achieves the lowest five-set mean recognition error among the compared streaming systems. The gains are more pronounced in speaker-attributed ASR: at an expected speaker-attribution latency of 2.00 s, VIBEVOICE-ASR-STREAMING achieves the best or tied-best cpWER/cpCER on 12 of the 13 evaluation settings, improving over Azure ConversationTranscriber by 2.39 to 12.45 points on the meeting benchmarks and from 27.06 to 22.75 on the MLC-Challenge average.

Following VibeVoice-ASR, we commit to comprehensive open-sourcing: the model weights and high-performance inference code with vLLM support are available through the links listed on the first page, together with an online demo.

VIBEVOICE-ASR-STREAMING has the following limitations.

- *Multilingual coverage:* Training requires word-level alignments produced by Qwen3-ForcedAligner-0.6B[Qwe26], so language coverage is limited by the aligner’s supported languages. VIBEVOICE-ASR-STREAMING is currently trained and evaluated on ten languages; broader coverage is left to future work.
- *Long-duration overlap:* VIBEVOICE-ASR-STREAMING handles short conversational overlaps, but performance degrades when multiple speakers overlap for long periods. This is because the decoder must serialize overlap speech into a single output stream. Separation-aware representations or multi-stream decoding may help.
- *Recording length:* The released checkpoints support recordings of up to eight minutes. Longer recordings require more context and therefore more computation, so the current limit is mainly due to compute cost rather than the model architecture.
- *First-packet latency:* The reported 2.00 s latency is a steady-state expectation. The first output requires one full chunk plus lookahead, giving 3.5 s at 22 frames and 2.5 s at 15. Reducing this startup delay without sacrificing the accuracy of larger chunks remains future work.

References

- [BDN⁺17] Hui Bu, Jiayu Du, Xingyu Na, Bengu Wu, and Hao Zheng. AISHELL-1: An open-source Mandarin speech corpus and a speech recognition baseline. In *20th Conference of the Oriental Chapter of the International Coordinating Committee on Speech Databases and Speech I/O Systems and Assessment (O-COCOSDA)*, pages 1–5, 2017.
- [CAB⁺05] Jean Carletta, Simone Ashby, Sebastien Bourban, Mike Flynn, Mael Guillemot, Thomas Hain, Jaroslav Kadlec, Vasilis Karaiskos, Wessel Kraaij, Melissa Kronenthal, et al. The ami meeting corpus: A pre-announcement. In *International workshop on machine learning for multimodal interaction*, pages 28–39. Springer, 2005.
- [CCW⁺21] Guoguo Chen, Shuzhou Chai, Guan-Bo Wang, Jiayu Du, Wei-Qiang Zhang, Chao Weng, Dan Su, Daniel Povey, Jan Trmal, Junbo Zhang, Mingjie Jin, Sanjeev Khudanpur, Shinji Watanabe, Shuaijiang Zhao, Wei Zou, Xiangang Li, Xuchen Yao, Yongqing Wang, Zhao You, and Zhiyong Yan. GigaSpeech: An evolving, multi-domain ASR corpus with 10,000 hours of transcribed audio. In *Interspeech 2021*, pages 3670–3674, 2021.
- [CHH⁺24] Zhehuai Chen, He Huang, Oleksii Hrinchuk, Krishna C. Puuvada, Nithin Rao Koluguri, Piotr Żelasko, Jagadeesh Balam, and Boris Ginsburg. BESTOW: Efficient and streamable speech language model with the best of two worlds in GPT and T5. *arXiv preprint arXiv:2406.19954*, 2024.
- [DLL⁺26] Yuhang Dai, Haopeng Lin, Zhennan Lin, Jiale Qian, Jun Wu, Hanke Xie, Hao Meng, Hanlin Wen, Chuang Ding, Shunshun Yin, Ming Tao, Lei Xie, and Xinsheng Wang. SoulX-Transcriber: A robust end-to-end framework for multi-speaker speech transcription. *arXiv preprint arXiv:2606.02400*, 2026.
- [FCL⁺21] Yihui Fu, Luyao Cheng, Shubo Lv, Yukai Jv, Yuxiang Kong, Zhuo Chen, Yanxin Hu, Lei Xie, Jian Wu, Hui Bu, et al. Aishell-4: An open source dataset for speech enhancement, separation, recognition and speaker diarization in conference scenario. *arXiv preprint arXiv:2104.03603*, 2021.
- [Gil26] GilgameshWind. X-ASR-zh-en. <https://huggingface.co/GilgameshWind/X-ASR-zh-en>, 2026. Model card, chunk-1920ms streaming configuration. Accessed August 2026.
- [HLS21] Eunjung Han, Chul Lee, and Andreas Stolcke. BW-EDA-EEND: Streaming end-to-end neural speaker diarization for a variable number of speakers. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7193–7197, 2021.
- [JKZ⁺24] Junteng Jia, Gil Keren, Wei Zhou, Egor Lakomkin, Xiaohui Zhang, Chunyang Wu, Frank Seide, Jay Mahadeokar, and Ozlem Kalinli. Efficient streaming LLM for speech recognition. *arXiv preprint arXiv:2410.03752*, 2024.
- [KWW⁺22a] Naoyuki Kanda, Jian Wu, Yu Wu, Xiong Xiao, Zhong Meng, Xiaofei Wang, Yashesh Gaur, Zhuo Chen, Jinyu Li, and Takuya Yoshioka. Streaming multi-talker ASR with token-level serialized output training. In *Interspeech 2022*, pages 3774–3778, 2022.
- [KWW⁺22b] Naoyuki Kanda, Jian Wu, Yu Wu, Xiong Xiao, Zhong Meng, Xiaofei Wang, Yashesh Gaur, Zhuo Chen, Jinyu Li, and Takuya Yoshioka. Streaming speaker-attributed ASR with token-level speaker embeddings. In *Interspeech 2022*, pages 521–525, 2022.
- [LKL21] Liang Lu, Naoyuki Kanda, Jinyu Li, and Yifan Gong. Streaming end-to-end multi-talker speech recognition. *IEEE Signal Processing Letters*, 28:803–807, 2021.
- [LSL24] Di Liang, Nian Shao, and Xiaofei Li. Frame-wise streaming end-to-end speaker diarization with non-autoregressive self-attention-based attractors. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 10521–10525, 2024.

- [MGS⁺25] Bingshen Mu, Pengcheng Guo, Zhaokai Sun, Shuai Wang, Hexin Liu, Mingchen Shao, Lei Xie, Eng Siong Chng, Longshuai Xiao, Qiangze Feng, et al. Summary on the multilingual conversational speech language model challenge: Datasets, tasks, baselines, and methods. *arXiv preprint arXiv:2509.13785*, 2025.
- [Mis26] Mistral AI. Voxtral realtime. *arXiv preprint arXiv:2602.11298*, 2026.
- [MPW⁺25] Ivan Medennikov, Taejin Park, Weiqing Wang, He Huang, Kunal Dhawan, Jinhan Wang, Jagadeesh Balam, and Boris Ginsburg. Streaming sortformer: Speaker cache-based online speaker diarization with arrival-time ordering. In *Interspeech 2025*, pages 5238–5242, 2025.
- [NVI26] NVIDIA. Nemotron 3.5 ASR streaming 0.6b. <https://huggingface.co/nvidia/nemotron-3.5-asr-streaming-0.6b>, 2026. Model card. Accessed August 2026.
- [PCL⁺26] Jing Peng, Ziyi Chen, Haoyu Li, Yucheng Wang, Duo Ma, Mengtian Li, Yunfan Du, Dezhu Xu, Kai Yu, and Shuai Wang. G-STAR: End-to-end global speaker-tracking attributed recognition. *arXiv preprint arXiv:2603.10468*, 2026.
- [PCPK15] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An ASR corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2015, South Brisbane, Queensland, Australia, April 19-24, 2015*, pages 5206–5210. IEEE, 2015.
- [PMD⁺25] Taejin Park, Ivan Medennikov, Kunal Dhawan, Weiqing Wang, He Huang, Nithin Rao Koluguri, Krishna C. Puvvada, Jagadeesh Balam, and Boris Ginsburg. Sortformer: A novel approach for permutation-resolved speaker supervision in speech-to-text systems. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [PYC⁺26] Zhiliang Peng, Jianwei Yu, Yaoyao Chang, Zilong Wang, Li Dong, Yingbo Hao, Yujie Tu, Chenyu Yang, Wenhui Wang, Songchen Xu, Yutao Sun, Hangbo Bao, Weijiang Xu, Yi Zhu, Zehua Wang, Ting Song, Yan Xia, Zewen Chi, Shaohan Huang, Liang Wang, Chuang Ding, Shuai Wang, Xie Chen, and Furu Wei. Vibevoice-asr technical report. *arXiv preprint arXiv:2601.18184*, 2026.
- [PYW⁺25] Zhiliang Peng, Jianwei Yu, Wenhui Wang, Yaoyao Chang, Yutao Sun, Li Dong, Yi Zhu, Weijiang Xu, Hangbo Bao, Zehua Wang, et al. Vibevoice technical report. *arXiv preprint arXiv:2508.19205*, 2025.
- [Qwe26] Qwen Team. Qwen3-forcedaligner-0.6b. <https://huggingface.co/Qwen/Qwen3-ForcedAligner-0.6B>, 2026. Model card. Accessed August 2026.
- [RPK23] Desh Raj, Daniel Povey, and Sanjeev Khudanpur. SURT 2.0: Advances in transducer-based multi-talker speech recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31:3800–3813, 2023.
- [RWM⁺24] Desh Raj, Matthew Wiesner, Matthew Maciejewski, Paola Garcia, Daniel Povey, and Sanjeev Khudanpur. On speaker attribution with SURT. In *The Speaker and Language Recognition Workshop (Odyssey 2024)*, pages 91–98, 2024.
- [SBW⁺24] Yutao Sun, Hangbo Bao, Wenhui Wang, Zhiliang Peng, Li Dong, Shaohan Huang, Jianyong Wang, and Furu Wei. Multimodal latent language modeling with next-token diffusion. *arXiv preprint arXiv:2412.08635*, 2024.
- [SXF⁺25] Mohan Shi, Xiong Xiao, Ruchao Fan, Shaoshi Ling, and Jinyu Li. Train short, infer long: Speech-LLM enables zero-shot streamable joint ASR and diarization on long audio. *arXiv preprint arXiv:2511.16046*, 2025.
- [vNBDHU23] Thilo von Neumann, Christoph Boeddeker, Marc Delcroix, and Reinhold Haeb-Umbach. MeetEval: A toolkit for computation of word error rates for meeting transcription systems. In *7th International Workshop on Speech Processing in Everyday Environments (CHiME 2023)*, pages 27–32, 2023.

- [XTH⁺26] Yinfeng Xia, Jian Tang, Junfeng Hou, Gaopeng Xu, and Haitao Yao. Uni-ASR: Unified LLM-based architecture for non-streaming and streaming automatic speech recognition. *arXiv preprint arXiv:2603.11123*, 2026.
- [YCD⁺25] Han Yin, Yafeng Chen, Chong Deng, Luyao Cheng, Hui Wang, Chao-Hong Tan, Qian Chen, Wen Wang, and Xiangang Li. Speakerlm: End-to-end versatile speaker diarization and recognition with multimodal large language models. *arXiv preprint arXiv:2508.06372*, 2025.
- [YLY⁺26] Donghua Yu, Zhengyuan Lin, Chen Yang, Yiyang Zhang, Zhaoye Fei, Hanfu Chen, Jingqi Chen, Ke Chen, Qinyuan Cheng, Liwei Fan, et al. Moss transcribe diarize: Accurate transcription with speaker diarization. *arXiv preprint arXiv:2601.01554*, 2026.
- [YYZ⁺24] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [YZF⁺22] Fan Yu, Shiliang Zhang, Yihui Fu, Lei Xie, Siqi Zheng, Zhihao Du, Weilong Huang, Pengcheng Guo, Zhijie Yan, Bin Ma, et al. M2met: The icassp 2022 multi-channel multi-party meeting transcription challenge. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6167–6171. IEEE, 2022.

A Output Format, Target Construction, and Training Details

The speaker label is written in plain text as `\n Speaker k;`; no timestamps are emitted, and a chunk containing no speech yields an empty Y_k . A word belongs to chunk k if its end time falls at or before the boundary of chunk k , within a 0.1 s tolerance.

The listing below shows an example output for the released 22-frame configuration, with one chunk arriving every 2.9 s.

```
chunk 0 : " \n Speaker 0:Okay, let's start with the deployment plan."
chunk 1 : " \n Speaker 1:Sure. I think we should first verify the latency "
chunk 2 : "on the meeting set. \n Speaker 0:Agreed. We can run that today."
chunk 3 : " \n Speaker 1:Do we also need to check the multilingual results?"
chunk 4 : ""
chunk 5 : " \n Speaker 0:Yes, especially French and German."
chunk 6 : " \n Speaker 1:Okay. I'll prepare the evaluation scripts, and "
chunk 7 : "then we can compare the numbers tomorrow morning."
chunk 8 : " \n Speaker 0:Sounds good. Please also save the per-file outputs."
chunk 9 : " \n Speaker 1:Will do. \n Speaker 0:Great."
chunk 10: "Let's review everything once the runs finish."
```

Stages 2 and 3 use AdamW ($\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1), gradient clipping at 2.0, bfloat16, a cosine schedule with peak learning rate 5×10^{-5} , and sequences packed to 8,192 tokens. Stage 2 is a multi-node run over the 420,000-hour corpus; Stage 3 is a 500-step run on eight GPUs at a global batch of 64 sequences with 35 warmup steps, and the released checkpoints are taken at step 400.

B Evaluation Protocol and Baseline Systems

Scoring. Chinese, Japanese, and Korean are scored at the character level, as CER and cpCER; every other language at the word level, as WER and cpWER. All systems are scored with the same normalization and the same MeetEval cpWER implementation, with every utterance counted.

Latency. VIBEVOICE-ASR-STREAMING is reported as the expected algorithmic delay $C/2 + T_{\text{lookahead}}$. For the cloud services we report a measured wall-clock delay:

$$\bar{\ell} = \frac{\sum_j n_j (\ell_j + d_j/2)}{\sum_j n_j}, \quad (4)$$

where ℓ_j is the delay between the end of the audio span covered by emitted result j and the time its speaker label settles, n_j and d_j are its word count and duration, and $d_j/2$ is the expected position of a word inside it. Word counts follow the scoring tokenization, and all baseline audio is pushed at no faster than $1 \times$ real time. Table 9 lists the side information each system receives.

Table 9: Side information supplied to each system at inference time. VIBEVOICE-ASR-STREAMING receives none of it.

System	Language prior	Speaker count	Hotwords / context
Azure CT	✓ explicit locale	–	–
Google STT	✓ en-US only	✓ oracle count	–
VIBEVOICE-ASR-STREAMING	–	–	–

Azure CT. ConversationTranscriber through the Python SDK `azure-cognitiveservices-speech` 1.51.1 against an S0-tier resource in eastus2. Each file runs in its own session from 16 kHz mono 16-bit PCM pushed through a `PushAudioInputStream`, and we consume only the final transcribed events. The language prior is zh-CN for AISHELL-4 and AliMeeting, en-US for AMI, and the per-utterance locale for MLC-Challenge, with pt-PT for Portuguese; no speaker count is supplied. The 8.21 s of Table 2 follows Equation 4.

Google STT. Google Cloud Speech-to-Text v1 (`google.cloud.speech_v1`) through the `StreamingRecognize` RPC, with LINEAR16 16 kHz mono audio under the `en-US` language code and `model` left unset. Interim results, automatic punctuation, word time offsets, and speaker diarization are enabled, with `min_speaker_count` set to 2 and `max_speaker_count` to the reference speaker count of each recording. Coverage is English only: AMI-IHM, AMI-SDM, and the English portion of MLC-Challenge, 30.80 h and 256,419 reference words in total. D_0 and D_{fin} carry identical text and differ only in speaker attribution, and the latency reported for Google STT times the settling of speaker labels rather than of text. Text settles after about 1.5 s, a word’s speaker label keeps changing for a further 16.5–31.3 s (per-recording medians), 42.6–67.9% of words have their label revised at least once after first emission, and within a 30 s revision budget 66.4% (AMI-IHM), 66.8% (AMI-SDM), and 78.4% (MLC English) of labels agree with D_{fin} .